COMP3361 Assignment 2 Report

Pang Tin Hei (3036100179)

April 6, 2026

1. Taggers

1.1 BiLSTM Tagger

The structure of the BiLSTM Tagger is as follows:

```
BiLSTMTagger(
  (embedding): Embedding(41326, 64, padding_idx=0)
  (dropout): Dropout(p=0.3, inplace=False)
  (lstm): LSTM(64, 64, num_layers=2, batch_first=True, dropout=0.3, bidirectional=True)
  (fc): Linear(in_features=128, out_features=37, bias=True)
)
```

The architecture consists of an `nn.Embedding` layer that maps vocabulary indices to fixed-size continuous vectors. The embeddings are passed through a dropout layer, then fed into a 2-layer Bidirectional Long Short-Term Memory (BiLSTM) network, which processes the sequence both forwards and backwards to capture bidirectional context. Finally, the BiLSTM hidden states are passed through another dropout layer and a linear fully connected layer (`nn.Linear`) to project them into logit scores for each entity label.

1.2 Transformer Tagger

The structure of the Transformer Tagger is as follows:

```
TransformerTagger(
  (embedding): Embedding(41326, 64, padding_idx=0)
  (pos_encoder): PositionalEncoding(
    (dropout): Dropout(p=0.0, inplace=False)
  )
  (transformer_encoder): TransformerEncoder(
    (layers): ModuleList(
      (0-3): 4 x TransformerEncoderLayer(
        (self_attn): MultiheadAttention(
          (out_proj): NonDynamicallyQuantizableLinear(in_features=64, out_features=64, bias=True)
        )
        (linear1): Linear(in_features=64, out_features=2048, bias=True)
        (dropout): Dropout(p=0.0, inplace=False)
        (linear2): Linear(in_features=2048, out_features=64, bias=True)
        (norm1): LayerNorm((64,), eps=1e-05, elementwise_affine=True)
        (norm2): LayerNorm((64,), eps=1e-05, elementwise_affine=True)
        (dropout1): Dropout(p=0.0, inplace=False)
        (dropout2): Dropout(p=0.0, inplace=False)
      )
    )
  )
  (fc): Linear(in_features=64, out_features=37, bias=True)
  (dropout): Dropout(p=0.0, inplace=False)
)
```

This model uses an embedding layer paired with static sinusoidal Positional Encoding to inject sequence order information, since self-attention inherently lacks sequential awareness. The sequence is fed into a Transformer Encoder composed of 4 identical layers, each with Multi-Head Self-Attention and feed-forward networks. The output is projected to the label size using a linear fully connected layer.

1.3 DistilBERT Tagger

This tagger fine-tunes a pre-trained `distilbert-base-cased` model. It utilizes DistilBERT's deep contextualized subword representations (WordPiece tokenization). The representations corresponding to the first subword of each token are mapped to entity classes using a linear classification head (`DistilBertForTokenClassification`).

2. Experimental Settings

The dataset is drawn from the OntoNotes 5.0 dataset, tokenized, and padded/truncated to a fixed maximum length (`MAX_LEN = 128`). The validation data in `valid.json` is used to evaluate model performance after each epoch, using the `seqeval` F1-score metric.

Each of the model comes with a Adam optimizer with a starting learning rate, and the loss function used is `CrossEntropyLoss` (ignoring padding indices where `label = -100`).

We have also tuned the hyperparameters for each model and observed their performance:

1. **BiLSTM:**

- Batch Size: 32
- Embedding Size
- Learning Rate: 10^{-3}
- Epochs: 5

2. **Transformer:**

- Batch Size: 32
- Embedding Size: 64
- Learning Rate: 10^{-3}
- Epochs: 10, 5
- Dropout Rate: 0.1, 0.3, 0.5

3. **DistilBERT:**

- Batch Size: 8, 4
- Learning Rate: 1×10^{-5}
- Epochs: 4, 3

3. Comparison Table (Test Set)

Model	Batch Size	Embedding Size	Learning Rate	Epochs	Dropout Rate	Test F1-score
BiLSTM	32	64	10^{-3}	5	0.3	0.7105
BiLSTM	32	64	10^{-2}	5	0.1	**0.7433**
Transformer	32	64	10^{-3}	10	0.5	0.1004
Transformer	32	64	10^{-3}	10	0.3	0.4045
Transformer	32	64	10^{-3}	5	0.1	0.5139
Transformer	32	64	10^{-3}	5	0.0	**0.5483**
DistilBERT	8	-	10^{-5}	4	-	**0.8639**
DistilBERT	4	-	10^{-5}	3	-	0.7856

4. Discussion and Analysis

The first observation is that the BiLSTM model significantly outperforms the Transformer model (0.7024 vs 0.4045) when trained from scratch on this dataset. It is largely due to the fact that transformers require massive amounts of

data to learn positional relationships and effective attention representations effectively. Without pre-training, the 4-layer Transformer struggles to generalize on a comparatively small NER dataset. On the other hand, the BiLSTM, structurally biased towards sequential data, is highly efficient at capturing local context even with limited training data.

Besides training from scratch, pre-trained models like DistilBERT exhibits great performance on the NER task, even with a small epoch training process. We think such performance can be attributed to the pre-training process, where the model has inherently captured complex syntax, semantics, and word contexts across diverse corpora. It mitigates the issue seen in the previous transformer model by effectively transferring contextual embeddings to the NER task. Therefore, DistilBERT requires less task-specific training time while yielding dominant F1 scores, proving the strength of the transfer-learning paradigm for Named Entity Recognition.

Additionally, the excessive time spent on training is also noticed, and the training time increases significantly with the model complexities. On my local machine, the BiLSTM model requires the least amount of time to train, while it takes around 30 minutes to train a DistilBERT model, even with only 3 epochs.

In summary, it is believed that the DistilBERT model possess the best performance among all models, and is able to offer the best generalizability to unseen data, due to its pre-trained nature.